Partner API — Plan & Best Practices

Status: Draft for review Scope: External REST API for **program partners only** (organizations that post opportunities and receive applications). Not a general public API. No admin, reviewer, or applicant surface.

---

1. Goal

Let a partner's own systems (CRM / ATS / grants portal) integrate with MAALI programmatically:

- Push and manage **their** opportunities.
- Read applications submitted to **their** opportunities.
- Receive real-time events via **webhooks**.

Hard constraint: a partner can only ever see or touch data belonging to their own `partner_id`. Enforced by the same Row Level Security (RLS) that backs the existing `/partner` dashboard.

---

2. System architecture

```
Partner system (CRM/ATS)
        │  HTTPS + JSON, X-API-Key
        ▼
Partner API  /v1   ── Supabase Edge Function (Deno) router
        │            - API-key auth → resolves partner_id
        │            - request validation (Zod)
        │            - rate limiting
        │            - audit logging
        ▼
PostgreSQL + RLS  ── scoped to partner_id (reuses existing policies)
        │
        ▼
Webhook dispatcher ── outbound POST to partner endpoint (HMAC signed)
```

Key points a tech lead will expect:

- **No raw PostgREST / anon key exposed to partners.** The public Supabase client stays internal to the web app. Partners only ever hit the `/v1` gateway.
- **Reuse, don't fork.** Auth helper (`supabase/functions/_shared/auth.ts`), CORS (`_shared/cors.ts`), rate limiting (pattern from `rate-limited-auth`), and existing RLS policies (`partner_id`, `is_partner_org_admin`) are reused.
- **Single source of truth for data model** is the Postgres schema in `src/integrations/supabase/types.ts`. The API exposes a stable, versioned subset — never the raw table shape.
- **Stateless gateway.** Auth + scoping derived per request from the API key. No server session.

---

3. Authentication & authorization

API keys

- Header: `X-API-Key: mpk_live_xxx` (prefix denotes env: `mpk_live_`, `mpk_test_`).
- Stored **hashed** (SHA-256) in a new `partner_api_keys` table; only prefix + last 4 shown after creation.
- Each key maps to exactly one `partner_id` and a set of scopes.
- Rotation and revocation supported (`revoked_at`).

Scopes

Scope	Grants
`opportunities:read`	List/get own opportunities
`opportunities:write`	Create/update/close own opportunities
`applications:read`	List/get applications to own opportunities (summary)
`applications:read_pii`	Include applicant PII + signed document URLs
`webhooks:manage`	Register/rotate webhook endpoint & secret

Proposed table

```sql
create table public.partner_api_keys (
  id           uuid primary key default gen_random_uuid(),
  partner_id   integer not null references public.partners(id) on delete cascade,
  key_hash     text not null,              -- SHA-256 of the secret
  key_prefix   text not null,              -- e.g. mpk_live_ab12 (display only)
  scopes       text[] not null default '{}',
  environment  text not null default 'test' check (environment in
('test','live')),
  last_used_at timestamptz,
  created_at   timestamptz not null default now(),
  revoked_at   timestamptz
);
create index on public.partner_api_keys (key_hash) where revoked_at is null;
```

RLS: table is service-role only. Key verification runs inside the edge function with the service role; partners never query this table.

---

4. Conventions (best practice)

Concern	Rule
Base URL	`https://api.<domain>/v1`
Versioning	Version in path (`/v1`). Breaking change → `/v2`. Additive changes stay in `/v1`.

Concern	Rule
Format	JSON only. `Content-Type: application/json`. UTF-8.
Auth	`X-API-Key` header. 401 if missing/invalid, 403 if scope insufficient.
Pagination	Cursor-based: `?limit=50&cursor=<opaque>`. Default 50, max 100. Response returns `next_cursor` (null when done).
Filtering	Explicit query params only (e.g. `?status=open`). No arbitrary query passthrough.
Idempotency	Writes accept `Idempotency-Key` header; duplicate key within 24h returns the original result.
Timestamps	ISO 8601 UTC (`2026-07-03T10:00:00Z`).
IDs	Opportunity id = integer. Application id = UUID string.
Errors	`{ "error": "message", "errorCode": "MACHINE_CODE" }` (matches existing edge-fn pattern).
Rate limit	60 req/min, 10,000 req/day per key. Returns `429` + `Retry-After`. Headers: `X-RateLimit-Remaining`, `X-RateLimit-Reset`.
Compatibility	Never remove/rename a field in `/v1`. Only add optional fields.

Standard error codes

HTTP	errorCode	Meaning
400	`VALIDATION_ERROR`	Body/params failed validation
401	`UNAUTHORIZED`	Missing/invalid API key
403	`FORBIDDEN_SCOPE`	Key lacks required scope
404	`NOT_FOUND`	Resource not found or not owned by partner
409	`IDEMPOTENCY_CONFLICT` / `CONFLICT`	Duplicate or state conflict
422	`UNPROCESSABLE`	Semantically invalid (e.g. deadline in past)
429	`RATE_LIMITED`	Too many requests
500	`INTERNAL_ERROR`	Unexpected server error

---

5. Data types

Derived from the live schema. The API uses **camelCase** externally; the gateway maps to snake_case columns internally.

Enums (exposed as-is)

```
opportunityType: accelerator | competition | fellowship | grant | hackathon |
                 internship | job | scholarship | training   (see schema for full
set)
fundingType:     fully_funded | partially_funded | stipend | no_funding | equity |
paid
experienceLevel: student | undergraduate | graduate | early_career | mid_career |
startup_founder
programFormat:   online | in_person | hybrid
opportunityStatus: open | closed | draft
applicationStatus: pending | under_review | approved | rejected
```

Opportunity (response)

Field	Type	Notes
`id`	integer	MAALI opportunity id
`title`	string	required
`description`	string	required
`status`	`opportunityStatus`	
`deadline`	string (ISO date)	required
`opportunityType`	`opportunityType` | null	
`fundingType`	`fundingType` | null	
`fundingAmount`	string | null	free-form (e.g. "10000")
`currency`	string | null	ISO 4217, default USD
`location`	string	
`country`	string | null	
`sectorId`	integer | null	
`experienceLevel`	`experienceLevel` | null	
`programFormat`	`programFormat` | null	
`requirements`	string | null	
`eligibilityCriteria`	string | null	
`maxApplicants`	integer | null	
`currentApplicants`	integer	read-only
`startDate`	string (ISO date) | null	
`endDate`	string (ISO date) | null	
`imageUrl`	string | null	

Field	Type	Notes
`featured`	boolean	read-only (platform-controlled)
`partnerId`	integer	always the caller's org
`createdAt` / `updatedAt`	string (ISO datetime)	read-only

Opportunity (create/update request)

Writable subset only. `id`, `currentApplicants`, `featured`, `partnerId`, timestamps are server-controlled and ignored if sent.

```json
{
  "title": "Agritech Accelerator 2026",
  "description": "12-week program for early-stage founders.",
  "status": "open",
  "deadline": "2026-09-30",
  "opportunityType": "accelerator",
  "fundingType": "equity",
  "fundingAmount": "50000",
  "currency": "USD",
  "location": "Accra, Ghana",
  "country": "GH",
  "sectorId": 3,
  "experienceLevel": "startup_founder",
  "programFormat": "hybrid",
  "requirements": "Registered business in West Africa.",
  "eligibilityCriteria": "Less than 3 years old, pre-Series A.",
  "maxApplicants": 200,
  "startDate": "2026-10-15",
  "endDate": "2027-01-15"
}
```

Application (summary — default)

Returned with `applications:read`. No PII beyond what the partner needs to rank.

Field	Type	Notes
`id`	string (UUID)	application id
`opportunityId`	integer	
`status`	`applicationStatus`	
`projectTitle`	string | null	
`projectSummary`	string | null	
`primarySectors`	array | null	
`averageScore`	number | null	from review system

Field	Type	Notes
`totalReviews`	integer	
`rankPosition`	integer | null	
`submittedAt`	string (ISO datetime)	
`createdAt` / `updatedAt`	string (ISO datetime)	

Application (PII fields — require `applications:read_pii`)

Only returned when the key holds `applications:read_pii`. Subject to data-processing consent and compliance review.

Field	Type
`fullLegalName`	string | null
`contactEmail`	string | null
`contactPhone`	string | null
`countryOfResidence`	string | null
`organizationName`	string | null
`linkedinUrl` / `githubUrl` / `otherSocialLinks`	string | null
`documentUrls`	array of short-lived signed URLs

---

6. Endpoints (v1)

Partner-only. Every route is implicitly scoped to the caller's `partner_id`.

Opportunities

Method	Path	Scope	Description
GET	`/v1/opportunities`	`opportunities:read`	List own opportunities (paginated, filter by `status`)
GET	`/v1/opportunities/{id}`	`opportunities:read`	Get one own opportunity
POST	`/v1/opportunities`	`opportunities:write`	Create (supports `Idempotency-Key`)
PATCH	`/v1/opportunities/{id}`	`opportunities:write`	Partial update
POST	`/v1/opportunities/{id}/close`	`opportunities:write`	Set status to `closed`

Applications

Method	Path	Scope	Description

Method	Path	Scope	Description
GET	`/v1/opportunities/{id}/applications`	`applications:read`	List ranked applications for an own opportunity
GET	`/v1/applications/{appId}`	`applications:read`	Get one application summary
GET	`/v1/applications/{appId}` (+PII)	`applications:read_pii`	Includes PII + signed document URLs

Organization

Method	Path	Scope	Description
GET	`/v1/organization`	`opportunities:read`	Get own org profile
PATCH	`/v1/organization`	`opportunities:write`	Update name, description, sector, website, logo

Webhooks (self-service)

Method	Path	Scope	Description
GET	`/v1/webhooks`	`webhooks:manage`	Get current endpoint + subscribed events
PUT	`/v1/webhooks`	`webhooks:manage`	Set endpoint URL + event list
POST	`/v1/webhooks/rotate-secret`	`webhooks:manage`	Rotate signing secret

---

7. Webhooks

Push model so the partner does not have to poll. This is the recommended integration path.

Events

Event	Fires when
`application.submitted`	Applicant submits (non-draft) to a partner opportunity
`application.status_changed`	Application moves between statuses
`opportunity.closed`	Opportunity reaches deadline or is closed

Delivery contract

- Transport: HTTPS POST, JSON body.

- Signature: `X-Maali-Signature: t=<unix>,v1=<hex>` where `v1 = HMAC_SHA256(secret, "<t>.<raw_body>")`.
- Replay protection: reject if `|now - t| > 5 min`.
- Partner must respond `2xx` within 5s.
- Retries: exponential backoff, 5 attempts (e.g. 1m, 5m, 30m, 2h, 6h), then dead-letter + internal alert.
- At-least-once delivery → **partner must dedupe on `event.id`**.

Payload

```json
{
  "id": "evt_2Zx...",
  "type": "application.status_changed",
  "createdAt": "2026-07-03T10:00:00Z",
  "data": {
    "applicationId": "b3f1...-uuid",
    "opportunityId": 42,
    "status": "approved",
    "previousStatus": "under_review"
  }
}
```

Verification (partner side, reference)

```ts
import crypto from "node:crypto";

function verify(rawBody: string, header: string, secret: string): boolean {
  const parts = Object.fromEntries(header.split(",").map((p) => p.split("=")));
  const expected = crypto
    .createHmac("sha256", secret)
    .update(`${parts.t}.${rawBody}`)
    .digest("hex");
  const ok = crypto.timingSafeEqual(Buffer.from(expected), Buffer.from(parts.v1));
  const fresh = Math.abs(Date.now() / 1000 - Number(parts.t)) < 300;
  return ok && fresh;
}
```

---

8. Transactions & pricing

Definition (adopt this)

> A **transaction** is one **successful inbound API request** (HTTP 2xx) from the partner to MAALI. Not counted: outbound webhooks, retries, and failed requests (4xx/5xx).

- **Webhooks are included** in the standard integration at no transaction cost. They are outbound and MAALI-initiated; billing them would penalize the efficient (low-load) pattern.

- Internal cost note: webhook sends are Supabase Edge Function invocations and count toward MAALI's own infra usage, but are **not** partner-billable transactions.

Volume estimate (single partner, webhook-first)

Activity	Assumption	Calls/month
Opportunity create/update	~20–50 writes	~40
Application reads (event-driven)	fetch detail on webhook	~200
Reconciliation poll	1×/hour, few endpoints	~2,200
Total (recommended pattern)		**~2,500**
If polling-only (no webhooks), hourly		**~15,000**

Headline for proposal: **~2,500–3,000 transactions/month (~30,000–35,000/year)** for one webhook-integrated partner; up to ~15k/month if they cannot accept webhooks and must poll.

---

9. Documentation deliverables

- OpenAPI 3.1 spec: `openapi/partner-v1.yaml` (source of truth).
- Hosted interactive docs (Redoc or Swagger UI).
- Auth & webhook guide (key issuance, rotation, signature verification).
- Postman collection generated from the spec.
- Error-code reference (Section 4).
- Changelog (semver).
- Sandbox (`mpk_test_`) + production (`mpk_live_`) environments.

---

10. Build plan

1. Migration: `partner_api_keys` (+ optional `partner_webhooks`, `partner_api_events` for audit/dead-letter).
2. Edge function `partner-api` (router) reusing `_shared/auth.ts` + new API-key verifier → resolves `partner_id`.
3. Zod validation per endpoint; map camelCase ↔ snake_case.
4. Read endpoints (opportunities, applications, organization) + pagination.
5. Write endpoints + idempotency.
6. Webhook dispatcher + signing + retry/dead-letter.
7. Rate limiting (reuse `rate-limited-auth` approach).
8. OpenAPI spec + hosted docs + Postman.
9. Audit logging per request.

Phasing

- **P1** — key auth + read endpoints + docs + sandbox.
- **P2** — write endpoints + idempotency.
- **P3** — webhooks + org update + PII scope.

11. Open item to confirm with partner

Only one thing that cannot be defaulted: **can the partner's system receive inbound webhooks, or is it outbound-only / firewalled?**

- Yes → webhook-first (recommended, ~2.5k tx/month).
- No → polling fallback (higher volume, define poll interval).

Everything else follows the best-practice defaults above.